Homework 2 : Planning for a High-DoF Arm

Gautam Balaji

October 12, 2026

1 Planners

RRT grows a single tree from the start configuration, with 10% of the samples set to the goal, until a node gets within 0.1 rad of the goal. The path is then shortened by skipping waypoints that can be connected directly.
RRT-Connect grows one tree from the start and one from the goal and tries to connect them after every extension. It is used without any path shortening.
RRT* grows the tree the same way as RRT, but picks the cheapest nearby parent for each new node and rewires nearby nodes through it. It runs for all 10,000 iterations and returns the cheapest path to the goal.
PRM builds a roadmap of 2000 nodes using Gaussian sampling, so nodes end up near obstacles, and connects each node to its 10 nearest neighbours. The start and goal are attached to the roadmap, and A* finds the path.

1.1 Hyper-parameters

The following hyperparameters are used in `planner.cpp`:

- Step Size : $0.5rad$
- Goal threshold : $0.1rad$
- num_samples : 10000

All the planners use these three hyperparameters by default. All edges are collision-checked every $\pi/90$ rad. RRT and RRT* use a goal bias of 0.1. RRT* uses a rewiring radius of 1.0 rad. The 10,000 iterations are what make RRT* slow, since it always runs all of them, but for PRM, 2000 samples were enough to connect the roadmap on map 2. The Gaussian sampling uses $\sigma = \pi/6$.

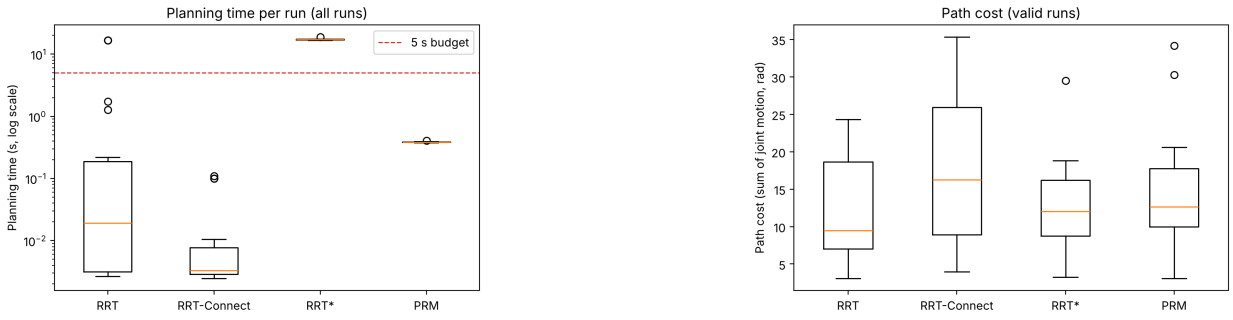
2 Results

The results below are the output of running each planner once on 20 randomly generated start and goal pairs on map2.txt with a 5 DOF arm. The pairs were generated once and fixed for all the planners. Path quality is the path cost used by the grader (sum of all joint angle movements), so lower is better.

Planner	Avg. planning time (s)	Success rate (under 5 s)	Avg. vertices	Avg. path cost
RRT	0.21	90% (18/20)	640	12.39
RRT-Connect	0.004	85% (17/20)	71	17.43
RRT*	17.33	0% (0/20)	10001	12.59
PRM	0.39	100% (20/20)	2002	14.38

Table 1: Results on map2.txt

In Table (1), planning time, vertices, and path cost are averaged over the runs that found a valid path. RRT* found a valid path for 17 of the 20 pairs, but never in under 5 seconds. RRT-Connect was by far the fastest, usually taking a few milliseconds, but it failed on 3 pairs and gave the worst paths, with about 40% higher cost than the others and many more waypoints. RRT was also fast on most pairs and gave short



(a) Planning time (log scale).

(b) Path cost.

Figure 1: Planning time and path cost for each planner over the 20 pairs.

paths because of the shortening step, but it failed on 2 pairs. RRT* gave the best paths, but it took about 17 seconds on every pair, so it never met the 5 second limit. PRM took about 0.4 seconds every time, almost all of it spent building the roadmap, and it was the only planner that solved all 20 pairs. Two of the pairs (6 and 11) could not be solved by any of the tree based planners, only by PRM. On the 16 pairs that all four planners solved, RRT*, RRT and PRM had almost the same path cost (11.5 to 12.0), while RRT-Connect was at 16.9. The number of vertices is fixed for RRT* and PRM, since RRT* always runs all its iterations and PRM always builds 2000 nodes, while for RRT and RRT-Connect it depends on how hard the pair is.

What planner do you think is the most suitable for the environment and why? PRM is the most suitable planner for this environment. It is the only planner that solved all 20 pairs, it always finished in under half a second, and its path cost on the common pairs is about the same as RRT and RRT*. Since its samples are placed near obstacles, it handles the narrow passages in map 2 that made the tree based planners fail.

What issues does the planner still have? The paths are not shortened, so they can be long on the harder pairs (34.2 against 24.3 for RRT on pair 4). The roadmap is rebuilt for every query, which is where almost all of its time goes. It uses a fixed number of samples, so if the start and goal end up in different parts of the roadmap it fails without trying again. Gaussian sampling also leaves few nodes in open areas, so paths through them go around the obstacles.

How do you think you can improve that planner? The same path shortening used for RRT can be applied to the PRM path. Some uniform samples can be mixed in with the Gaussian ones to cover the open areas, and more samples can be added when the start and goal are not connected instead of failing. The roadmap can also be kept and reused across queries since the map does not change.

3 Time until First Solution for RRT*

RRT* found its first solution after 0.146 s on average (median 0.003 s, maximum 2.02 s on pair 4), compared to its total run time of 17.33 s. On 16 of the 17 pairs it solved, the first solution came in under 0.2 s, which is about the same as RRT (0.21 s). So RRT* spends about 99% of its time improving a path it already has, and that only makes the path about 3% cheaper than the shortened RRT path.

4 Sampling Heuristics for RRT*

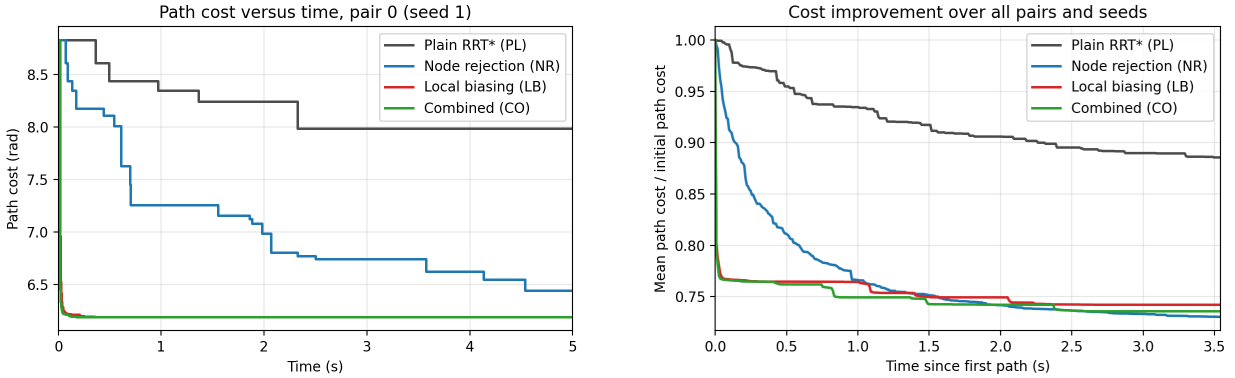
The SAMPLE() function of RRT* was modified to include the sampling heuristics from Akgun and Stilman. Both heuristics are only switched on after the first path is found, so with the same random seed all methods find the same first path and only differ in how they improve it.

- **Node Rejection (NR):** a sample is thrown away if the straight line distance from the start to the sample plus from the sample to the goal is more than the cost of the current best path, since it cannot make the path cheaper.
- **Local Biasing (LB):** 30% of the samples ($\beta = 0.3$) are taken near a random node of the current path, moved towards the middle of its two neighbours by a random length between 0.1 and 2 step sizes, which straightens the path.
- **Combined (CO):** both NR and LB together.

Vanilla RRT* (PL) and the three heuristics were run on 3 of the 20 pairs, with 10 fixed seeds per method and a time budget of 5 s per run, the same way as in the paper. The cost here is the Euclidean path length in joint space, which is the cost RRT* minimises. The heuristic version is selected with extra command line arguments after the usual 6 (heuristic, time budget and seed), so the default call used by the grader is unchanged.

Pair	Method	c_1	c_{end}	Cost decrease	Iterations
0	PL	9.99 (2.10)	7.53 (0.34)	22.0%	82,355
	NR	9.99 (2.10)	6.41 (0.09)	33.4%	2,763,974
	LB	9.99 (2.10)	6.40 (0.29)	33.7%	37,320
	CO	9.99 (2.10)	6.40 (0.29)	33.7%	50,085
1	PL	5.42 (0.52)	5.22 (0.29)	3.4%	82,932
	NR	5.42 (0.52)	4.61 (0.01)	14.4%	16,157,481
	LB	5.42 (0.52)	4.59 (0.06)	14.8%	42,417
	CO	5.42 (0.52)	4.59 (0.06)	14.8%	137,159
2	PL	5.38 (0.79)	4.62 (0.49)	13.1%	84,324
	NR	5.38 (0.79)	3.36 (0.05)	36.5%	8,482,492
	LB	5.38 (0.79)	3.60 (0.18)	31.9%	60,564
	CO	5.38 (0.79)	3.60 (0.18)	31.9%	74,584

Table 2: RRT* Performance within 5 s, averaged over 10 runs



(a) Path cost versus time for a single run (pair 0, seed 1).

(b) Mean path cost (relative to the first path) over all runs.

Figure 2: Path cost versus time for vanilla RRT* and the sampling heuristics.

In Table (2), parentheses indicate standard deviation. c_1 is the cost of the first path and c_{end} the cost at the end of the time budget. On pair 2, 2 of the 10 seeds did not find a first path within 5 s for any method. All three heuristics roughly doubled the cost decrease of vanilla RRT* in the same 5 s. Over all pairs and seeds, the cost decreased by 12.8% for PL, 27.5% for NR, and 26.4% for both LB and CO. Local biasing gave the fastest improvement: LB and CO made the path 20% cheaper within about 0.01 s of finding it, while vanilla RRT* took about 1.6 s to do the same on the runs where it got there at all. After that, LB and CO level off, since they keep straightening the same path towards a local optimum. Node rejection improved more

slowly but kept improving until the end of the time budget and finished with the lowest average cost. It also ran many more iterations, since rejecting a sample is much cheaper than extending the tree. On pair 1, the path is close to a straight line, so almost every sample was rejected, and the tree only grew to 1,708 nodes compared to 5,751 for vanilla RRT*. CO ended with exactly the same cost as LB on every seed, because the samples NR throws away are never part of the best path. Still, it reached that cost faster (its last improvement came after 0.75 s on average, against 1.25 s for LB), since it does not waste time extending the tree towards them.